AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Założenia konstrukcyjne i funkcjonalne programu 3

1.2. Wymagania dotyczące systemu kontroli wersji Git 4

2. Analiza problemu **5**

2.1. Budowa i zasada działania listy dwukierunkowej 5

2.2. Rola wzorców projektowych w rozwiązaniu problemu 6

2.3. Zestawienie operacji i ich złożoności obliczeniowej 6

3. Projektowanie **7**

3.1. Wykorzystane narzędzia i biblioteki 7

3.2. Ustawienia kompilatora i budowanie projektu 8

3.3. Architektura plików i diagram klas UML 8

3.4. Schemat działania algorytmu wstawiania pod indeks 9

3.5. Sposób używania narzędzi Git, GitHub oraz Doxygen 10

4. Implementacja **11**

4.1. Mechanizm przepinania wskaźników w pamięci 11

4.2. Implementacja węzła i wzorca Factory 12

4.3. Implementacja klasy listy dwukierunkowej 13

4.4. Implementacja wzorca Iteratora i testów w funkcji main 14

5. Wnioski **16**

Literatura **17**

Spis rysunków **17**

Spis tabel **18**

Spis listingów **19**

1. Ogólne określenie wymagań

Celem projektu jest napisanie w języku C++ programu obsługującego listę dwukierunkową, której elementy są alokowane dynamicznie na stercie. Równorzędnym celem zadania jest poznanie w praktyce systemu kontroli wersji Git (w połączeniu z serwisem GitHub) oraz nauka automatycznego generowania dokumentacji technicznej za pomocą programu Doxygen.

1.1. Założenia konstrukcyjne i funkcjonalne programu

Projekt zakłada pełne podejście obiektowe oraz ścisłą organizację plików źródłowych. Podczas pisania kodu należało spełnić trzy główne założenia architektoniczne:

- **Podział na osobne pliki:** każda klasa wchodząca w skład programu musi być rozdzielona na plik nagłówkowy (`.h`) z deklaracjami oraz plik źródłowy (`.cpp`) z właściwą implementacją metod.
- **Działanie na stercie:** lista oraz wszystkie jej węzły muszą być tworzone w pamięci dynamicznej za pomocą operatora `new` i ręcznie zwalniane operatorem `delete`, aby uniknąć wycieków pamięci.
- **Wykorzystanie wzorców projektowych:** w implementacji należało zastosować wzorzec fabryki (*Factory*) do tworzenia obiektów oraz wzorzec iteratora (*Iterator*) do poruszania się po strukturze.

Zaprojektowana klasa listy dwukierunkowej musiała zostać wyposażona w konkretny zestaw metod pozwalających na pełną manipulację danymi:

- dodawanie nowego elementu na sam początek oraz na sam koniec listy,
- wstawianie nowego elementu pod dowolny, wskazany przez użytkownika indeks,
- usuwanie elementu z początku, z końca oraz spod wskazanego indeksu,
- wyświetlanie zawartości całej listy od początku do końca oraz w odwrotnej kolejności (od końca do początku),
- wyświetlanie następnego oraz poprzedniego elementu względem aktualnej pozycji za pomocą iteratora,
- całkowite czyszczenie listy wraz ze zwolnieniem pamięci zajmowanej przez węzły.

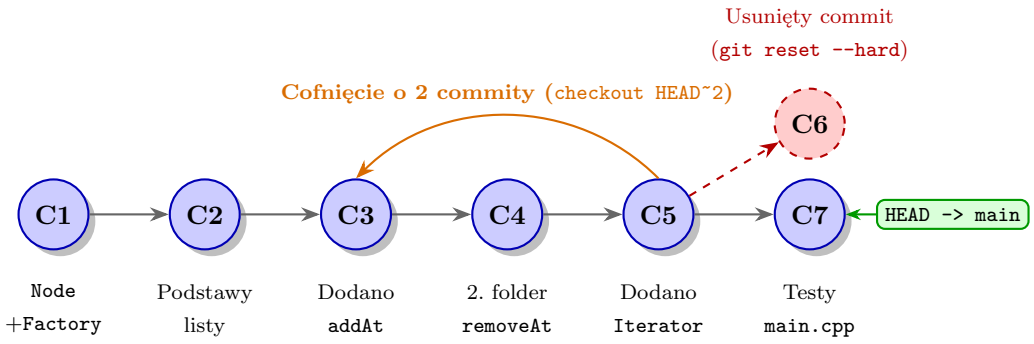
Poprawność działania wszystkich wyżej wymienionych metod należało w pełni przetestować w funkcji głównej `main`.

1.2. Wymagania dotyczące systemu kontroli wersji Git

Praca nad kodem nie polegała na wgraniu gotowego projektu na raz, lecz na systematycznym zapisywaniu postępów w zdalnym repozytorium na platformie GitHub. Instrukcja projektowa wymagała przetestowania kilku praktycznych scenariuszy, które zdarzają się w codziennej pracy programisty.

Na początku wykonano serię co najmniej pięciu commitów dokumentujących kolejne etapy powstawania klas. Następnie przetestowano cofanie się w historii zmian o dwa commity wstecz, aby podejrzeć wcześniejszy stan kodu, oraz całkowite usunięcie jednego błędnego commita. Kolejnym zadaniem było zasymulowanie pracy zespołowej (lub pracy na dwóch urządzeniach) – projekt pobrano z GitHuba do zupełnie innego folderu na dysku, dopisano w nim nową metodę, wysłano zmiany na serwer, a następnie pobrano je w pierwszym folderze i kontynuowano pracę. Ostatni scenariusz polegał na celowym usunięciu jednego z plików źródłowych, co doprowadziło do błędu kompilacji, a następnie uratowaniu projektu przez pobranie ostatniej poprawnej wersji z repozytorium.

Cały opisany wyżej proces pracy z historią zmian został zobrazowany na rysunku 1.1.



Rys. 1.1. Graficzny schemat historii commitów i operacji wykonanych w systemie Git

Jak widać na powyższym schemacie, główna linia rozwoju projektu przebiegała przez commity od `C1` do `C5`, uwzględniając zmianę katalogu roboczego przy commicie `C4`. Pomarańczowa strzałka obrazuje moment testowego cofnięcia się w historii o dwa kroki wstecz (do stanu z commita `C3`), natomiast czerwony węzeł `C6` przedstawia próbny commit, który został trwale wycofany z gałęzi przed dodaniem ostatecznych testów w `C7`.

2. Analiza problemu

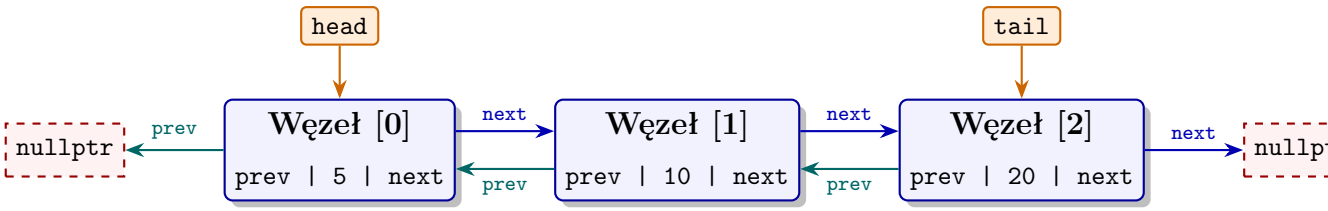
Klasyczne tablice w języku C++ zajmują jeden ciągły obszar w pamięci. Takie rozwiązanie ma dużą wadę – rozmiar tablicy jest stały, a próba wstawienia nowego elementu na jej początek lub w środek wymaga przesunięcia wszystkich pozostałych elementów, co jest bardzo nieefektywne. Problem ten rozwiązuje dynamiczna lista dwukierunkowa, w której elementy mogą znajdować się w różnych miejscach pamięci na stercie i są ze sobą połączone za pomocą wskaźników.

2.1. Budowa i zasada działania listy dwukierunkowej

Podstawową cegiełką budującą listę jest węzeł (klasa `Node`). W przeciwieństwie do listy jednokierunkowej, gdzie można poruszać się tylko do przodu, węzeł listy dwukierunkowej posiada dwa wskaźniki kierunkowe. Każdy taki obiekt przechowuje w pamięci trzy pola:

- **Wartość (`data`):** właściwa informacja przechowywana w węźle (w naszym projekcie liczba całkowita `int`).
- **Wskaźnik `next`:** przechowuje adres pamięci kolejnego węzła na liście (Dla ostatniego elementu ma wartość `nullptr`).
- **Wskaźnik `prev`:** przechowuje adres pamięci poprzedniego węzła (Dla pierwszego elementu ma wartość `nullptr`).

Cała lista jest zarządzana przez klasę nadrzędną (`DoublyLinkedList`), która nie musi znać adresów wszystkich elementów – wystarczy, że pamięta wskaźnik na pierwszy element (`head`) oraz ostatni element (`tail`). Sposób łączenia poszczególnych węzłów za pomocą wskaźników w pamięci dynamicznej przedstawiono na rysunku 2.1.



Rys. 2.1. Architektura węzłów i wskaźników listy dwukierunkowej w pamięci dynamicznej

Analizując powyższy schemat, można zauważyć, że każdy węzeł środkowy posiada dokładne informacje o obu swoich sąsiadach. Z kolei skrajne wskaźniki (wskaźnik `prev` pierwszego węzła oraz wskaźnik `next` ostatniego węzła) wskazują na war-

tość pustą `nullptr`, co jest dla programu sygnałem, że osiągnięto początek lub koniec listy podczas jej przeglądania.

2.2. Rola wzorców projektowych w rozwiązaniu problemu

Aby kod programu był przejrzysty i zgodny z zasadami programowania obiektowego, wykorzystano w nim dwa wzorce projektowe:

- **Wzorzec Factory (Fabryka):** rozwiązuje problem rozproszenia operatora `new` po całym kodzie programu. Zamiast alokować pamięć ręcznie wewnątrz metod listy lub w funkcji `main`, zadanie to zlecono dedykowanej klasie `ListFactory`. Posiada ona statyczne metody wytwórcze, które tworzą gotowe obiekty na stercie i zwracają do nich wskaźniki.

- **Wzorzec Iterator:** rozwiązuje problem bezpiecznego przeglądania listy krok po kroku. Gdyby użytkownik w funkcji `main` sam operował bezpośrednio na wskaźnikach węzłów, mógłby łatwo uszkodzić strukturę listy. Klasa `ListIterator` ukrywa w sobie wskaźnik `current` i udostępnia proste metody do przejścia o jeden element w przód (`showNext`) lub w tył (`showPrev`).

2.3. Zestawienie operacji i ich złożoności obliczeniowej

Poszczególne operacje na liście dwukierunkowej różnią się od siebie liczbą kroków, które musi wykonać program. W tabeli 2.1 zebrano wszystkie metody zaimplementowane w projekcie wraz z określeniem ich złożoności czasowej oraz opisem sposobu działania.

Jak wynika z tabeli 2.1, największą zaletą naszej listy dwukierunkowej są operacje wykonywane na jej początku oraz końcu (`addFront`, `addBack`, `removeFront`, `removeBack`). Dzięki temu, że klasa zawsze pamięta wskaźniki `head` oraz `tail`, metody te mają złożoność stałą $O(1)$ – wykonują się natychmiastowo, niezależnie od tego, czy na liście znajduje się pięć, czy dziesięć tysięcy elementów. Operacje wymagające dostępu do konkretnego indeksu w środku listy lub wyświetlenia wszystkich elementów mają złożoność liniową $O(n)$, ponieważ program musi przejść po kolejnych wskaźnikach w pętli.

Tab. 2.1. Opis i złożoność operacji na liście dwukierunkowej

Metoda / Operacja	Złożoność	Opis działania w programie
`addFront` / `addBack`	$O(1)$	Tworzy nowy węzeł przez fabrykę i od razu podpina go pod wskaźnik `head` lub `tail`.
`removeFront` / `removeBack`	$O(1)$	Przesuwa `head` lub `tail` na sąsiedni element i zwalnia stary węzeł operatorem `delete`.
`addAt` / `removeAt`	$O(n)$	Wymaga przejścia pętlą od początku listy do wybranego indeksu i przepięcia wskaźników sąsiadów.
`displayForward` / `Backward`	$O(n)$	Przechodzi w pętli przez całą listę od `head` (przez `next`) lub od `tail` (przez `prev`) i wypisuje dane.
`showNext` / `showPrev`	$O(1)$	Przesuwa wskaźnik iteratora `current` o jeden węzeł w prawo lub w lewo i wyświetla jego wartość.
`clear`	$O(n)$	Wywołuje w pętli usuwanie pierwszego elementu aż do całkowitego zwolnienia pamięci ze sterty.

3. Projektowanie

3.1. Wykorzystane narzędzia i biblioteki

Do zaprojektowania, napisania oraz udokumentowania programu wykorzystano zestaw darmowych narzędzi programistycznych dostępnych w środowisku Linux:

- **Język programowania C++:** wybrany ze względu na wsparcie dla programowania obiektowego oraz możliwość niskopoziomowego zarządzania pamięcią na stercie za pomocą wskaźników i operatorów `new` oraz `delete`.
- **Środowisko Visual Studio Code:** edytor kodu źródłowego wyposażony w rozszerzenie *C/C++ Extension Pack* oraz wbudowany moduł do obsługi repozytoriów Git.
- **Kompilator `g++` oraz debugger `gdb`:** narzędzia z pakietu GCC służące do kompilacji wieloplikowego projektu oraz śledzenia wartości wskaźników w pamięci podczas szukania błędów.

- **Biblioteka standardowa `<iostream>`:** jedyna biblioteka zewnętrzna dołączona do projektu, wykorzystywana do obsługi strumienia wyjścia (`cout`) podczas wyświetlania zawartości listy. Cała logika struktury danych została napisana od zera, bez używania gotowych kontenerów z biblioteki STL.
- **System Git, GitHub oraz Doxygen:** narzędzia wykorzystane odpowiednio do kontroli wersji kodu, przechowywania repozytorium w chmurze oraz automatycznego wygenerowania dokumentacji technicznej do formatu PDF.

3.2. Ustawienia kompilatora i budowanie projektu

Ponieważ wymogiem zadania było umieszczenie każdej klasy w osobnym pliku, projekt składa się łącznie z dziewięciu plików (czterech plików nagłówkowych `.h` oraz pięciu plików źródłowych `.cpp`). Próba standardowego skompilowania samego pliku `main.cpp` kończy się błędem linkera (`undefined reference`), ponieważ kompilator musi połączyć ze sobą implementacje wszystkich klas jednocześnie.

W tym celu skonfigurowano proces budowania w pliku `tasks.json` tak, aby wywoływał kompilator `g++` dla wszystkich plików źródłowych naraz:

```
g++ -g main.cpp Node.cpp DoublyLinkedList.cpp ListFactory.cpp
ListIterator.cpp -o program
```

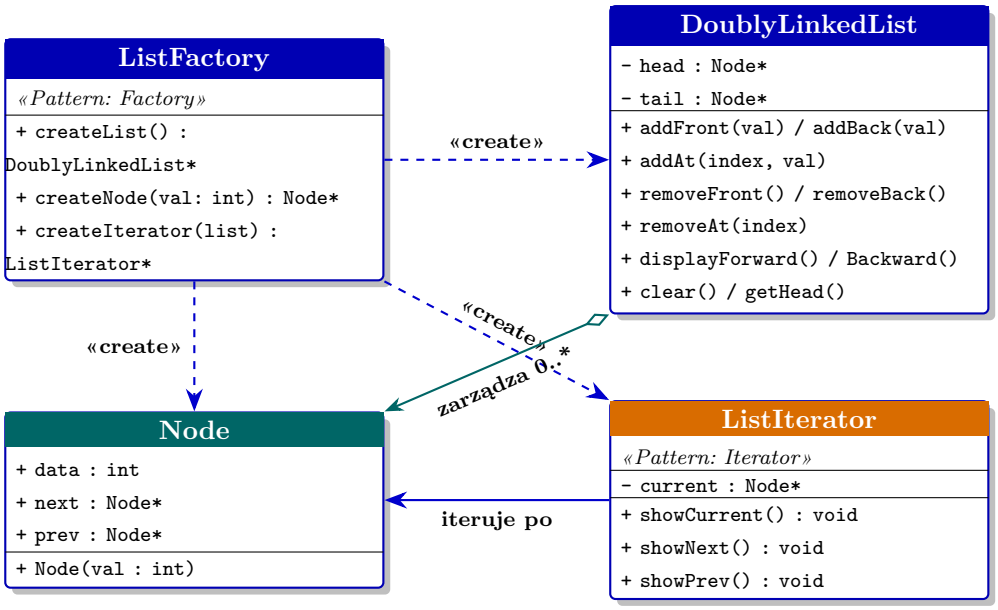
Zastosowana flaga `-g` nakazuje kompilatorowi dołączenie tablicy symboli debugowania, co umożliwia pracę z debuggerem `gdb`, natomiast przełącznik `-o program` określa nazwę wynikowego pliku wykonywalnego. Dodatkowo w każdym pliku nagłówkowym (`.h`) zastosowano dyrektywy preprocesora `#ifndef`, `#define` oraz `#endif` (tzw. strażników nagłówka), które zapobiegają błędowi wielokrotnego definiowania tej samej klasy podczas łączenia plików.

3.3. Architektura plików i diagram klas UML

Przed przystąpieniem do implementacji dokładnie rozplanowano podział obowiązków pomiędzy poszczególne pliki projektu. Szczegółowe zestawienie wszystkich plików wraz z opisem ich roli przedstawiono w tabeli **??**.

Zależności pomiędzy opisanymi w tabeli **??** klasami oraz dokładny wykaz ich pól prywatnych i metod publicznych zostały przedstawione na diagramie klas UML na rysunku 3.1.

Diagram na rysunku 3.1 pokazuje, że klasa `ListFactory` pełni rolę centralnego punktu tworzenia obiektów (relacje `<<create>>` oznaczone przerywanymi li-



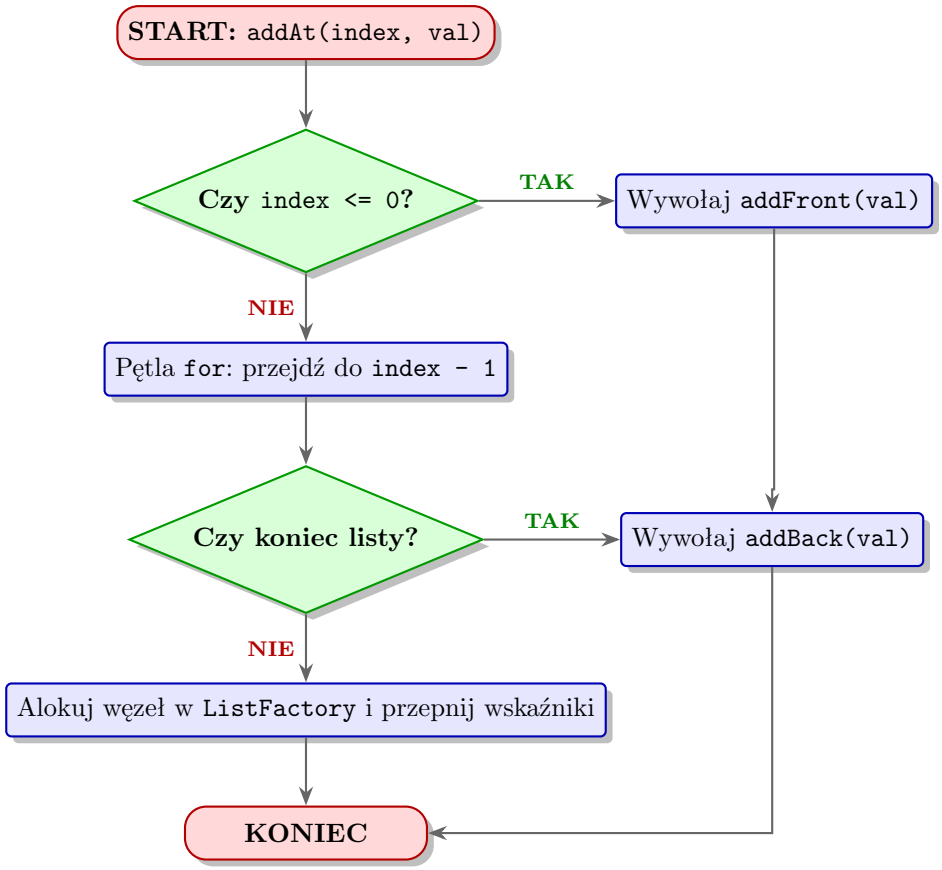
Rys. 3.1. Diagram klas UML z uwzględnieniem wzorców projektowych

niami). Klasa `DoublyLinkedList` agreguje wiele obiektów klasy `Node`, przechowując je w postaci powiązanego łańcucha, natomiast `ListIterator` operuje bezpośrednio na wskaźnikach węzłów, pobierając punkt startowy z listy za pomocą metody `getHead()`.

3.4. Schemat działania algorytmu wstawiania pod indeks

Jedną z najbardziej złożonych operacji w projekcie jest dodawanie nowego elementu pod wskazany indeks (metoda `addAt`), ponieważ wymaga ona uwzględnienia kilku przypadków brzegowych. Logikę tego algorytmu rozrysowano na schemacie blokowym na rysunku 3.2.

Zgodnie ze schematem na rysunku 3.2, algorytm najpierw sprawdza, czy podany indeks jest mniejszy bądź równy zero – jeśli tak, element jest od razu dodawany na początek metodą `addFront`. W przeciwnym razie pętla przesuwa wskaźnik pomocniczy do pozycji poprzedzającej docelowe miejsce (`index - 1`). Jeżeli w trakcie tego przejścia lista się skończy (użytkownik podał indeks większy niż rozmiar listy), element zostaje bezpiecznie dopisany na sam koniec metodą `addBack`. Dopiero gdy indeks wskazuje na środek listy, tworzony jest nowy węzeł i następuje przepięcie czterech wskaźników łączących go z sąsiadami.



Rys. 3.2. Schemat blokowy algorytmu wstawiania elementu pod wskazany indeks

3.5. Sposób używania narzędzi Git, GitHub oraz Doxygen

Praca z systemem Git oraz platformą GitHub

Obsługa systemu kontroli wersji odbywała się z poziomu terminala oraz wtyczki w VS Code. Na początku w folderze projektu zainicjowano lokalne repozytorium komendą `git init` i połączono je ze zdalnym serwerem na GitHubie za pomocą polecenia `git remote add origin <adres_URL>`. Po napisaniu każdej kolejnej części kodu pliki dodawano do poczekalni poleceniem `git add .`, a następnie zatwierdzano w historii komendą `git commit -m "opis zmian"`. Aby wysłać gotowe commity na serwer GitHub, używano polecenia `git push`, natomiast do pobrania zmian wprowadzonych w drugim katalogu służyła komenda `git pull`. Podczas testowania scenariuszy awaryjnych wykorzystano polecenia `git checkout HEAD~2` (cofnięcie się o dwa commity), `git reset --hard HEAD~1` (usunięcie ostatniego commita) oraz `git checkout -- Node.cpp` (odzyskanie usuniętego pliku z repozytorium).

Generowanie dokumentacji w programie Doxygen

Aby automatycznie wygenerować dokumentację z kodu, w plikach nagłówkowych (`.h`) zastosowano specjalne bloki komentarzy rozpoczynające się od znaków `/**`, wewnątrz których umieszczono znaczniki `@brief` (krótki opis klasy lub metody)

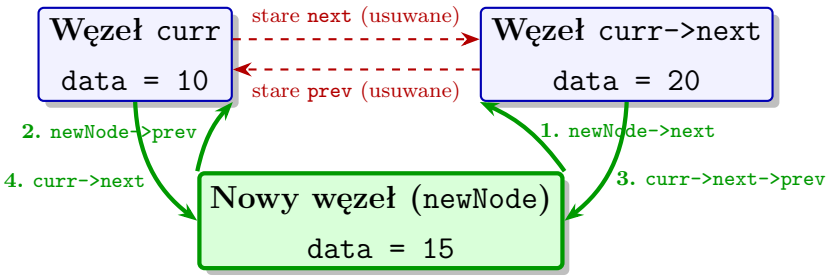
oraz `@param` (opis parametrów funkcji). Następnie w terminalu wywołano komendę `doxygen -g`, która utworzyła plik konfiguracyjny `Doxyfile`. W pliku tym upewniono się, że opcja `GENERATE_LATEX` jest ustawiona na `YES`. Kolejnym krokiem było uruchomienie analizy kodu poleceniem `doxygen Doxyfile`, co spowodowało automatyczne utworzenie podkatalogu `latex`. Po wejściu do tego katalogu wpisano polecenie `make`, które skompilowało wygenerowane pliki TeX bezpośrednio do gotowego dokumentu `refman.pdf`.

4. Implementacja

W niniejszym rozdziale omówiono szczegółową implementację poszczególnych klas w języku C++ oraz przedstawiono kluczowe fragmenty kodu źródłowego.

4.1. Mechanizm przepinania wskaźników w pamięci

Najważniejszym zagadnieniem podczas implementacji listy dwukierunkowej jest zachowanie odpowiedniej kolejności przepinania wskaźników. Jeśli wskaźnik na dalszą część listy zostanie nadpisany zbyt wcześnie, utracimy adresy kolejnych elementów i dojdzie do wycieku pamięci. Dokładną kolejność czterech operacji wykonywanych podczas wstawiania nowego węzła (o wartości `15`) pomiędzy węzeł `10` a `20` przedstawiono na rysunku 4.1.



Rys. 4.1. Kolejność przepinania 4 wskaźników podczas wstawiania nowego węzła w środek listy

Jak pokazano na rysunku 4.1, w krokach 1 oraz 2 najpierw ustawiane są wskaźniki nowego węzła (`newNode->next` oraz `newNode->prev`), aby wskazywały na swoich przyszłych sąsiadów. Dopiero gdy nowy węzeł jest już połączony z listą, w kroku 3 zmieniany jest wskaźnik `prev` prawego sąsiada, a na samym końcu (krok 4) aktualizowany jest wskaźnik `curr->next` lewego sąsiada.

4.2. Implementacja węzła i wzorca Factory

Pierwszym etapem kodowania było stworzenie klasy `Node` oraz współpracującej z nią fabryki `ListFactory`. Zawartość plików nagłówkowych obu tych klas zaprezentowano na listingu 1.

```
// --- Plik Node.h ---
#ifndef NODE_H
#define NODE_H

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int val);
};
#endif

// --- Plik ListFactory.h ---
#ifndef LISTFACTORY_H
#define LISTFACTORY_H

#include "DoublyLinkedList.h"
#include "ListIterator.h"
#include "Node.h"

class ListFactory {
public:
    static DoublyLinkedList* createList();
    static Node* createNode(int val);
    static ListIterator* createIterator(DoublyLinkedList* list);
};
#endif
```

Listing 1. Pliki nagłówkowe Node.h oraz ListFactory.h

W powyższym kodzie (listing 1) wszystkie pola klasy `Node` zostały zadeklarowane jako publiczne, aby klasa listy oraz iterator miały bezpośredni dostęp do wskaźników `next` i `prev`. Z kolei w klasie `ListFactory` zastosowano słowo kluczowe `static` przy wszystkich trzech metodach wytwórczych. Dzięki temu w pozostałych częściach programu możemy wywoływać np. `ListFactory::createNode(val)` bezpośrednio po nazwie klasy, bez konieczności tworzenia osobnego obiektu fabryki. W pliku źródłowym `ListFactory.cpp` każda z tych metod wywołuje operator `new`, rezerwując

pamięć na stercie.

4.3. Implementacja klasy listy dwukierunkowej

Główna logika zarządzania wskaźnikami została umieszczona w pliku `DoublyLinkedList.cpp`. Na listingu 2 przedstawiono kod trzech reprezentatywnych metod: dodawania na początek (`addFront`), dodawania pod wskazany indeks (`addAt`) oraz usuwania spod indeksu (`removeAt`).

```
void DoublyLinkedList::addFront(int val) {
    Node* newNode = ListFactory::createNode(val);
    if (!head) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
}

void DoublyLinkedList::addAt(int index, int val) {
    if (index <= 0) {
        addFront(val);
        return;
    }
    Node* curr = head;
    for (int i = 0; i < index - 1 && curr; i++) {
        curr = curr->next;
    }
    if (!curr || !curr->next) {
        addBack(val);
        return;
    }
    Node* newNode = ListFactory::createNode(val);
    newNode->next = curr->next;
    newNode->prev = curr;
    curr->next->prev = newNode;
    curr->next = newNode;
}

void DoublyLinkedList::removeAt(int index) {
    if (!head) return;
    if (index <= 0) {
        removeFront();
```

```
36          return;
37      }
38      Node* curr = head;
39      for (int i = 0; i < index && curr; i++) {
40          curr = curr->next;
41      }
42      if (!curr) return;
43      if (!curr->next) {
44          removeBack();
45          return;
46      }
47      curr->prev->next = curr->next;
48      curr->next->prev = curr->prev;
49      delete curr;
50  }
```

Listing 2. Wybrane metody z pliku DoublyLinkedList.cpp

W metodzie `addFront` (listing 2) warunek `if (!head)` sprawdza, czy lista jest obecnie pusta. Jeśli tak, nowo utworzony węzeł staje się jednocześnie głową (`head`) i ogonem (`tail`) listy. W metodzie `removeAt` po znalezieniu odpowiedniego węzła w pętli `for`, sąsiadujące z nim elementy zostają spięte ze sobą bezpośrednio (`curr->prev->next = curr->next`), omijając usuwany węzeł. Na samym końcu instrukcja `delete curr` zwalnia pamięć zajmowaną przez ten węzeł na stercie.

4.4. Implementacja wzorca Iteratora i testów w funkcji main

Ostatnim elementem kodu jest klasa `ListIterator` oraz plik główny `main.cpp`, których implementację przedstawiono na listingu 3.

```
1  // --- Plik ListIterator.cpp ---
2  #include "ListIterator.h"
3  #include <iostream>
4  using namespace std;
5
6  ListIterator::ListIterator(Node* startNode) : current(startNode) {}
7
8  void ListIterator::showCurrent() const {
9      if (current) cout << "Aktualny element: " << current->data <<
       endl;
10     else cout << "Iterator jest pusty." << endl;
11 }
12
13 void ListIterator::showNext() {
```

```
14      if (current && current->next) {
15          current = current->next;
16          cout << "Nastepny element: " << current->data << endl;
17      } else {
18          cout << "Brak nastepnego elementu (koniec listy)." << endl;
19      }
20  }
21
22  void ListIterator::showPrev() {
23      if (current && current->prev) {
24          current = current->prev;
25          cout << "Poprzedni element: " << current->data << endl;
26      } else {
27          cout << "Brak poprzedniego elementu (poczatek listy)." <<
        endl;
28      }
29  }
30
31  // --- Plik main.cpp ---
32  #include <iostream>
33  #include "ListFactory.h"
34  using namespace std;
35
36  int main() {
37      DoublyLinkedList* list = ListFactory::createList();
38
39      list->addBack(10);
40      list->addBack(20);
41      list->addFront(5);
42      list->addAt(2, 15);
43      list->displayForward();
44      list->displayBackward();
45
46      ListIterator* it = ListFactory::createIterator(list);
47      it->showCurrent();
48      it->showNext();
49      it->showPrev();
50
51      list->removeAt(2);
52      list->removeFront();
53      list->removeBack();
54      list->displayForward();
55
56      list->clear();
57      delete it;
```

```
58      delete list;
59      return 0;
60  }
```

Listing 3. Implementacja iteratora (ListIterator.cpp) oraz funkcja main.cpp

W klasie `ListIterator` (listing 3) metody `showNext()` oraz `showPrev()` zostały zabezpieczone warunkami sprawdzającymi, czy kolejny lub poprzedni węzeł w ogóle istnieje (`current->next != nullptr`). Chroni to program przed błędem naruszenia ochrony pamięci (tzw. *segmentation fault*) przy próbie wyjścia poza zakres listy. W funkcji `main` przetestowano po kolei wszystkie metody wymagane w instrukcji, a na końcu zwolniono ze sterty zarówno obiekt iteratora (`delete it`), jak i samą listę (`delete list`).

5. Wnioski

Realizacja projektu pozwoliła na praktyczne przećwiczenie programowania obiektowego w języku C++ oraz obsługi narzędzi wykorzystywanych w codziennej pracy programisty.

- **Działanie listy dwukierunkowej na stercie:** samodzielna implementacja wskaźników `next` oraz `prev` pozwoliła dokładnie zrozumieć, jak działają dynamiczne struktury danych. Bardzo ważne okazało się pilnowanie zwalniania pamięci operatorem `delete` przy każdym usuwaniu węzła oraz w destruktorze listy.
- **Wzorce projektowe i podział na pliki:** rozbicie programu na osobne pliki `.h` oraz `.cpp` wymusiło odpowiednią konfigurację kompilatora `g++`, ale sprawiło, że kod stał się znacznie bardziej przejrzysty. Zastosowanie wzorca *Factory* uporządkowało tworzenie obiektów na stercie, a wzorzec *Iterator* ułatwił przeglądanie kolejnych i poprzednich elementów listy.
- **Praca z systemem Git i serwisem GitHub:** przetestowane scenariusze pokazały, jak bardzo przydatny jest system kontroli wersji. Nawet w przypadku przypadkowego usunięcia ważnego pliku z kodem lub wprowadzenia błędu, Git pozwala w kilka sekund cofnąć się do poprzedniej działającej wersji programu oraz wygodnie synchronizować pracę między różnymi folderami.
- **Dokumentacja w Doxygen i LaTeX:** użycie programu Doxygen pokazało, że odpowiednio opisany kod źródłowy pozwala wygenerować gotową dokumentację techniczną niemal automatycznie, co oszczędza bardzo dużo czasu.

Spis rysunków

1.1. Graficzny schemat historii commitów i operacji wykonanych w systemie Git 4
2.1. Architektura węzłów i wskaźników listy dwukierunkowej w pamięci dynamicznej 5
3.1. Diagram klas UML z uwzględnieniem wzorców projektowych 9
3.2. Schemat blokowy algorytmu wstawiania elementu pod wskazany indeks 10
4.1. Kolejność przepinania 4 wskaźników podczas wstawiania nowego węzła w środek listy 11

Spis tabel

2.1. Opis i złożoność operacji na liście dwukierunkowej 7

Spis listingów

1. Pliki nagłówkowe Node.h oraz ListFactory.h 12
2. Wybrane metody z pliku DoublyLinkedList.cpp 13
3. Implementacja iteratora (ListIterator.cpp) oraz funkcja main.cpp . . . 14